



**Mondragon
Unibertsitatea**

**Escuela Politécnica
Superior**

Reinforcement Learning -RL-

Temporal Difference Learning

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

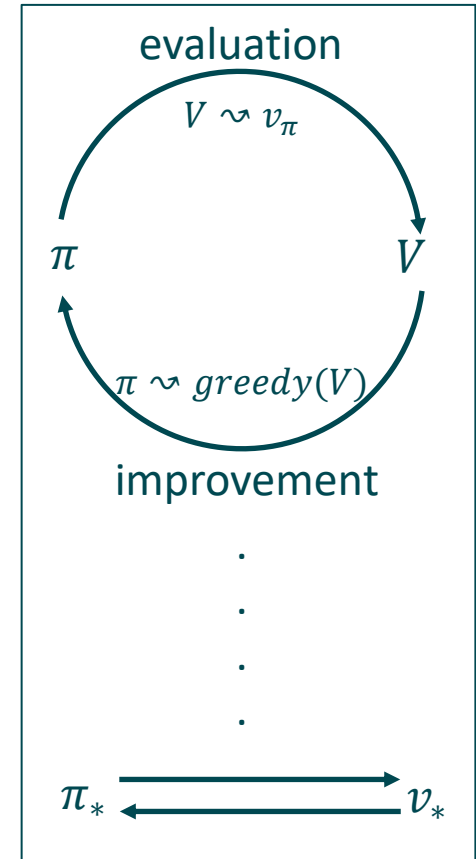
www.mondragon.edu

This presentation is based on:

Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction.
MIT Press, Cambridge, MA, 2018.

Temporal Difference Learning

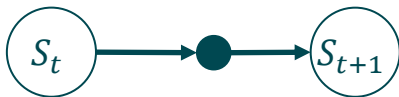
- **Value-based method**
 - **GPI (Generalized Policy Iteration):**
 - All we need is to find how to update the value function.



Temporal difference learning

- TD methods use experience to solve the prediction problem.
- TD methods need to wait only **until the next time step**.
- TD is model free: no knowledge of MDP transitions/rewards
- The simplest TD method makes the update:

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{\left[\overbrace{R_{t+1} + \gamma V(S_{t+1})}^{\text{TD target}} - V(S_t) \right]}_{\substack{\delta_t \\ \text{TD error}}}$$



Temporal difference learning

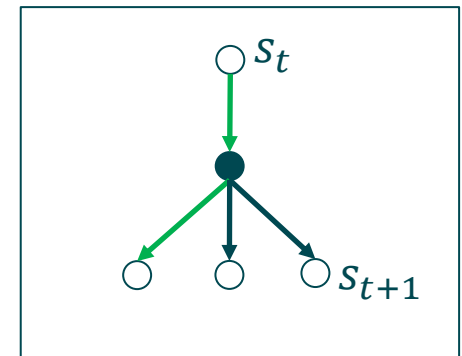
- But are TD methods sound? Can we be sure that we will get the right answer? **Fotunately, YES!!**
 - If you have a fixed policy π , it has been proved that TD(0) will eventually get to v_π .
- If both TD and Monte Carlo methods converge asymptotically to the correct predictions. But... which method learns faster?
 - In practice, TD methods have usually been found to work out faster than constant α MC methods. But this still is an open question...

Temporal difference learning

- We want to improve our estimate of V by computing these averages.

$$V_{k+1}(s) \leftarrow \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

- Sample 1: $R(s, a, s') + \gamma V_k(s')$

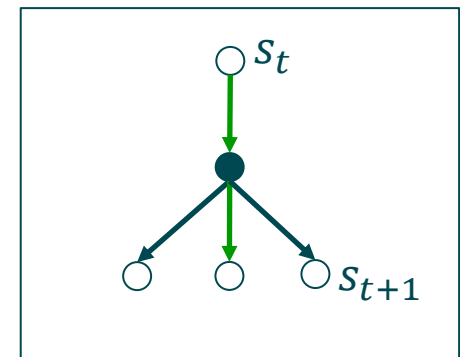


Temporal difference learning

- We want to improve our estimate of V by computing these averages.

$$V_{k+1}(s) \leftarrow \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

- Sample 1: $R(s, a, s') + \gamma V_k(s')$
- Sample 2: $R(s, a, s') + \gamma V_k(s')$

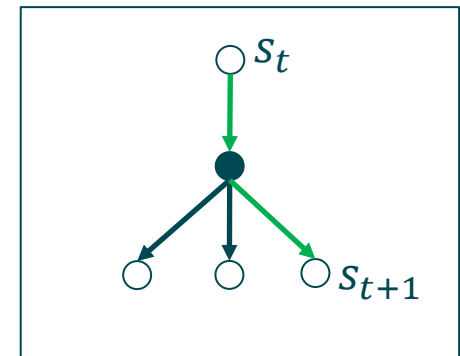


Temporal difference learning

- We want to improve our estimate of V by computing these averages.

$$V_{k+1}(s) \leftarrow \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

- Sample 1: $R(s, a, s') + \gamma V_k(s')$
- Sample 2: $R(s, a, s') + \gamma V_k(s')$
- Sample 3: $R(s, a, s') + \gamma V_k(s')$



$$V_{k+1}(s_t) = \sum sample_i$$

We cannot go back in time to get samples

Temporal difference learning

- We can use weighted format to update the new value function:

$$\begin{aligned} V(s) &= (1 - \alpha)V(s) + (\alpha)sample \\ &= V(s) + \alpha(sample - V(s)) \end{aligned}$$

- α : is a kind of learning rate

Temporal difference learning

- Model free
- Online learning
 - Can be implemented to continuing tasks
- Better convergence time
 - In practice, converge faster than Monte Carlo method
- TD can learn before knowing the outcome
- TD can learn without the final outcome
 - TD can learn from incomplete sequences
 - MC can only learn from complete sequences
 - TD works in continuing (non-terminating) environments
 - MC only works for episodic (terminating) environments

Temporal difference learning

- General pseudocode

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

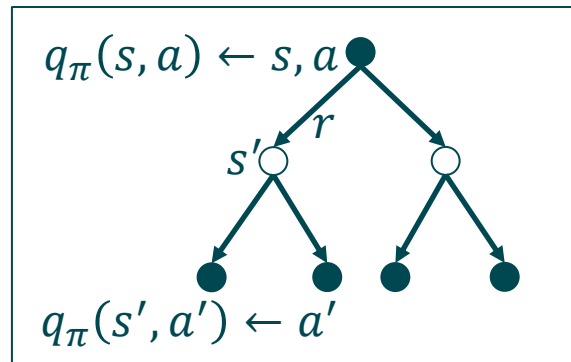
$S \leftarrow S'$

 until S is terminal

Temporal difference learning

- **Sarsa**
 - Previously we have introduced TD learning to predict the value function by one step sample.
 - Sarsa is inspired by policy iteration, and **we replace value function by action value function.**
 - The behaviour policy and target policy is the same

$$Q_{\pi}(s, a) \leftarrow \sum_{s'} P(s'|s, \pi(s)) [R(s, \pi(s), s') + \gamma Q_{\pi}(s', \pi(s'))]$$



Temporal difference learning

- Sarsa update**

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R(s, a, s') + \gamma Q(s', a') - Q(s, a)]$$



Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

On-policy

Temporal difference learning

$$\alpha = 0.1$$

$$\varepsilon = 0.1$$

$$\gamma = 0.9$$

• SARSA Episode 1:

- Episode History: [(S, Right, -1), (A, Left, -1), (S, Right, -1), (A, Down, 10)]; W=1

S {-1}	A {-1}
B {-1}	G {+10}

$$\begin{aligned}
 Q(S, right) &= Q(S, right) + 0.1[(-1) + 0.9Q(A, left) - Q(S, right)] & 1 \\
 Q(S, right) &= 0 + 0.1[(-1) + 0.9(0) - 0] = -0.1
 \end{aligned}$$

$$\begin{aligned}
 Q(A, left) &= Q(A, left) + 0.1[(-1) + 0.9Q(S, right) - Q(A, left)] & 2 \\
 Q(A, left) &= 0 + 0.1[(-1) + 0.9(-0.1) - 0] = -0.19
 \end{aligned}$$

$$\begin{aligned}
 Q(S, right) &= Q(S, right) + 0.1[(-1) + 0.9Q(A, down) - Q(S, right)] & 3 \\
 Q(S, right) &= -0.1 + 0.1[(-1) + 0.9(0) + 0.1] = -0.19
 \end{aligned}$$

$$\begin{aligned}
 Q(A, down) &= Q(A, down) + 0.1[(10) + 0.9G - Q(A, down)] & 4 \\
 Q(A, down) &= 0 + 0.1[(10) + 0.9(0) + 0] = 1
 \end{aligned}$$

Q

State	Right	Left	Down
S	-0.19	0	0
A	0	-0.19	1
B	0	0	0

Temporal difference learning

$$\alpha = 0.1$$

$$\varepsilon = 0.1$$

$$\gamma = 0.9$$

• SARSA Episode 2:

- Episode History: [(S, Right, -1), (A, Down, 10)]; W=1

1
2

S {-1}	A {-1}
B {-1}	G {+10}

$$Q(S, right) = Q(S, right) + 0.1[(-1) + 0.9Q(A, Down) - Q(S, right)] \quad \mathbf{1}$$

$$Q(S, right) = -0.19 + 0.1[(-1) + 0.9(1) + 0.19] = -0.181$$

$$Q(A, down) = Q(A, down) + 0.1[(10) + 0.9(0) - Q(A, down)] \quad \mathbf{2}$$

$$Q(A, down) = 1 + 0.1[(10) + 0.9(0) - 1] = 1.9$$

Q

State	Right	Left	Down
S	-0.181	0	0
A	0	-0.19	1.9
B	0	0	0

Temporal difference learning

$$\alpha = 0.1$$

$$\varepsilon = 0.1$$

$$\gamma = 0.9$$

• SARSA Episode 3:

- Episode History: [(S, Down, -1), (B, right, 10)]; $W=1$

S {-1}	A {-1}
B {-1}	G {+10}

$$Q(S, down) = Q(S, down) + 0.1[(-1) + 0.9Q(B, right) - Q(S, down)] \quad 1$$

$$Q(S, down) = 0 + 0.1[(-1) + 0.9(0) + 0] = -0.1$$

$$Q(B, right) = Q(B, right) + 0.1[(10) + 0.9(0) - Q(B, right)] \quad 2$$

$$Q(A, down) = 0 + 0.1[(10) + 0.9(0) - 0] = 1$$

Q

State	Right	Left	Down
S	-0.198	0	-0.1
A	0	-0.19	1.8
B	1	0	0

Temporal difference learning

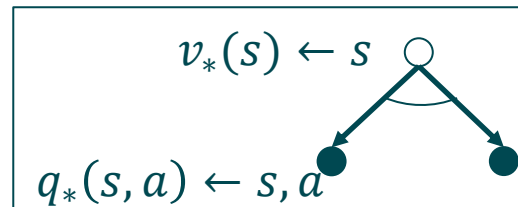
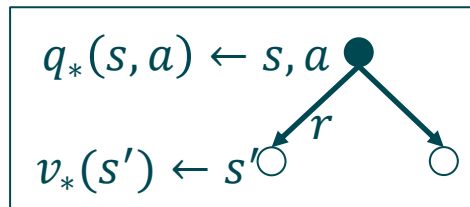
- **Q-learning**

- Inspired from value iteration

- In this case, the learned action-value function, Q , directly approximates q_* , the optimal action-value function, independent of the policy being followed.

$$Q(s, a) \leftarrow \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

$$v_*(s) = \max_a q_*(s, a)$$



Temporal difference learning

- Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

Off-policy

$S \leftarrow S'$

 until S is terminal

Temporal difference learning

$$\alpha = 0.1$$

$$\varepsilon = 0.1$$

$$\gamma = 0.9$$

• Q-learning Episode 1:

- **Episode History:** [(S, Right, -1), (A, Left, -1), (S, Right, -1), (A, Down, 10)]; W=1

S {-1}	A {-1}
B {-1}	G {+10}

$$\begin{aligned}
 Q(S, right) &= Q(S, right) + 0.1[(-1) + 0.9Q_{max}(A) - Q(S, right)] & \mathbf{1} \\
 Q(S, right) &= 0 + 0.1[(-1) + 0.9(0) - 0] = -0.1
 \end{aligned}$$

$$\begin{aligned}
 Q(A, left) &= Q(A, left) + 0.1[(-1) + 0.9Q_{max}(S) - Q(A, left)] & \mathbf{2} \\
 Q(A, left) &= 0 + 0.1[(-1) + 0.9(0) - 0] = -0.1
 \end{aligned}$$

$$\begin{aligned}
 Q(S, right) &= Q(S, right) + 0.1[(-1) + 0.9Q_{max}(S) - Q(S, right)] & \mathbf{3} \\
 Q(S, right) &= -0.1 + 0.1[(-1) + 0.9(0) + 0.1] = -0.199
 \end{aligned}$$

$$\begin{aligned}
 Q(A, down) &= Q(A, down) + 0.1[(10) + 0.9Q_{max}(G) - Q(A, down)] & \mathbf{4} \\
 Q(A, down) &= 0 + 0.1[(10) + 0.9(0) + 0] = 1
 \end{aligned}$$

Q

State	Right	Left	Down
S	-0.19	0	0
A	0	-0.1	1
B	0	0	0

Temporal difference learning

$$\alpha = 0.1$$

$$\varepsilon = 0.1$$

$$\gamma = 0.9$$

- Q-learning Episode 2:

- Episode History: [(S, Right, -1), (A, Down, 10)]; W=1

1
2

S {-1}	A {-1}
B {-1}	G {+10}

$$Q(S, right) = Q(S, right) + 0.1[(-1) + 0.9Q_{max}(A) - Q(S, right)] \quad 1$$

$$Q(S, right) = -0.19 + 0.1[(-1) + 0.9(1) + 0.19] = -0.18$$

$$Q(A, down) = Q(A, down) + 0.1[(10) + 0.9Q_{max}(G) - Q(A, down)] \quad 2$$

$$Q(A, down) = 1 + 0.1[(10) + 0.9(0) - 1] = 1.9$$

Q

State	Right	Left	Down
S	-0.18	0	0
A	0	-0.1	1.9
B	0	0	0

Temporal difference learning

$$\alpha = 0.1$$

$$\varepsilon = 0.1$$

$$\gamma = 0.9$$

- Q-learning Episode 3:

- Episode History: [(S, Down, -1), (B, right, 10)]; W=1

1
2

S {-1}	A {-1}
B {-1}	G {+10}

$$Q(S, down) = Q(S, down) + 0.1[(-1) + 0.9Q_{max}(B) - Q(S, down)] \quad 1$$

$$Q(S, down) = 0 + 0.1[(-1) + 0.9(0) + 0] = -0.1$$

$$Q(B, right) = Q(B, right) + 0.1[(10) + 0.9Q_{max}(G) - Q(B, right)] \quad 2$$

$$Q(B, right) = 0 + 0.1[(10) + 0.9(0) - 0] = 1$$

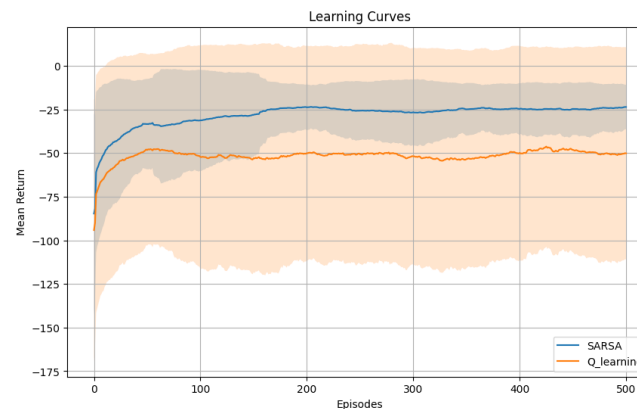
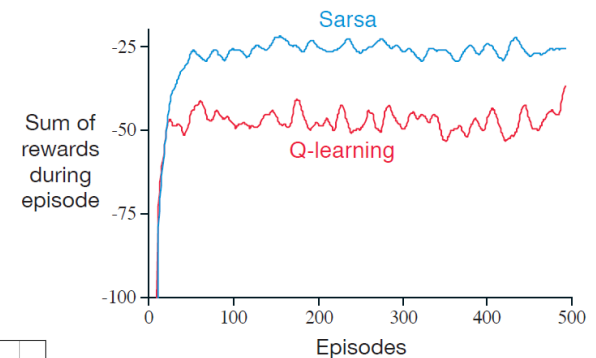
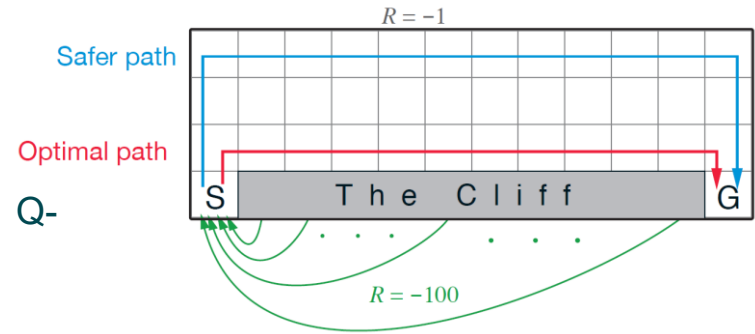
Q

State	Right	Left	Down
S	-0.18	0	-0.1
A	0	-0.1	1.9
B	1	0	0

Temporal difference learning

- **The Cliff Walking example:**

- This gridworld example compares Sarsa and Q-learning. ($\epsilon = 0.1$)
- Q-learning **learns values for the best policy**, which takes you right along the edge of the cliff. Unfortunately, this means it sometimes **falls off the cliff because of the ϵ -greedy action selection**.
- Sarsa, on the other hand, **considers the action selection** and learns the longer but safer path through the upper part of the grid.



Temporal difference learning

- Temporal-difference (TD) learning is introduced as a method for solving reinforcement learning problems, divided into prediction and control problems.
- TD methods serve as alternatives to Monte Carlo methods for the prediction problem and are extended to the control problem via generalized policy iteration (GPI).
- GPI involves two processes: one for accurately predicting returns (prediction problem) and the other for locally improving the policy (control problem).
- TD control methods can be classified into on-policy (e.g., Sarsa) and off-policy (e.g., Q-learning, Expected Sarsa) approaches.
- TD methods are widely used due to their simplicity, allowing online processing with minimal computation, and can be extended to more powerful forms in future chapters.



**Mondragon
Unibertsitatea**

Escuela Politécnica
Superior

Eskerrik asko
Muchas gracias
Thank you

© 2026 Mondragon Unibertsitatea.

All rights reserved. This document and its contents are protected by intellectual property rights. Its total or partial reproduction, distribution, public communication, transformation, adaptation or creation of derivative works is expressly prohibited without the written consent of the owner.

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu